

Technical Documentation

RAG-Based Customer Support Assistant

System Design & Implementation Guide

1. Introduction

What is RAG?

Retrieval-Augmented Generation (RAG) is a technique that combines information retrieval with language generation. Instead of relying only on pre-trained knowledge, the model retrieves relevant data from external sources and uses it to generate accurate responses.

Why It Is Needed

Traditional chatbots rely on static responses or pre-trained knowledge, which may not be up-to-date or domain-specific. RAG improves accuracy by grounding responses in real data sources such as documents.

Use Case Overview

This project implements a customer support assistant that answers user queries based on a PDF knowledge base. The system:

- Retrieves relevant information from the document corpus
- Generates contextual, grounded responses
- Escalates complex queries to human agents when required

2. System Architecture

Overall Architecture

The system is designed as a Retrieval-Augmented Generation (RAG) pipeline with an integrated workflow engine. It consists of two main stages: document ingestion and query processing.

Stage	Description
Document Ingestion	Processes input PDF by extracting text, chunking, and generating vector embeddings stored in ChromaDB.
Query Processing	Converts user queries into embeddings, retrieves relevant chunks, and passes them to the LLM as context.
Workflow Orchestration	Managed by LangGraph for controlled, structured execution with conditional routing.
Decision & Routing	Evaluates response quality post-generation and routes to user or

	escalates.
Human-in-the-Loop (HITL)	Escalates to human agent when confidence is low; response is delivered back to user.

3. Design Decisions

Chunk Size Selection

- A moderate chunk size balances context and retrieval accuracy.
- Smaller chunks improve precision but may lose context.
- Larger chunks provide more context but reduce retrieval relevance.
- Optimal chunk size: 500-1000 tokens for effective information retrieval.

Embedding Strategy

- Semantic embeddings used instead of keyword-based matching.
- Retrieves context based on meaning rather than exact word matches.
- Significantly improves the quality and relevance of responses.

Retrieval Approach

- Top-k similarity search used to fetch the most relevant chunks.
- Ensures only the most useful context is passed to the LLM.
- Reduces noise and improves response accuracy.

Prompt Design Logic

- Prompt includes both the user query and retrieved context.
- Helps the LLM generate grounded and context-aware responses.
- Prevents hallucinated answers by anchoring responses in real data.

Workflow Design Choice

- Graph-based workflow using LangGraph instead of a linear pipeline.
- Enables conditional routing and better control over decision-making.
- Supports complex state transitions and Human-in-the-Loop integration.

4. Workflow Explanation

The system follows a structured workflow with clearly defined stages:

Step 1 — Query Input

The workflow begins when a user submits a query through the interface. The query is received and prepared for processing.

Step 2 — Retrieval Process

The query is converted into an embedding and passed to the retriever. The retriever searches the vector database and fetches the most relevant document chunks based on cosine similarity.

Step 3 — Response Generation

Retrieved context is combined with the user query and passed to the LLM. The model generates a context-aware response using this grounded information.

Step 4 — Workflow Execution

The generated response is passed through the LangGraph workflow engine. The workflow ensures proper sequencing of operations and enables dynamic decision-making.

Step 5 — State Transitions

A shared state object carries data — including the query, retrieved context, and generated response — between different nodes in the workflow graph.

Step 6 — Final Output

Based on decision logic, the system either returns the generated response directly to the user or escalates the query to a human agent.

5. Conditional Logic

Intent Detection

The system analyzes the user query to understand its intent. Queries related to known topics are processed normally; ambiguous or out-of-scope queries are flagged for further evaluation.

Response Evaluation

After the LLM generates a response, the system evaluates its quality based on:

- Relevance of retrieved context
- Completeness of the answer
- Clarity and coherence of the response

Decision Making

The system applies conditional logic to determine the next step:

- Response is relevant and complete → Return to user
- Response is uncertain or incomplete → Escalate to HITL

Escalation Triggers

Escalation is triggered under the following conditions:

- Confidence score falls below threshold
- Retrieved context is insufficient or irrelevant
- Query is complex or outside the knowledge base

6. Human-in-the-Loop (HITL)

Role of Human Intervention

HITL plays a critical role in handling cases where the system is unable to generate a confident or accurate response. It ensures that users receive reliable answers even when the AI system encounters limitations.

When HITL Is Triggered

HITL is activated under the following conditions:

- Low confidence in the generated response
- Insufficient or irrelevant retrieved context
- Complex or ambiguous user queries
- Queries that fall outside the knowledge base

Process of Human Intervention

When escalation is triggered:

- The user query is forwarded to a human agent
- The system may include retrieved context or a partial response
- The human agent reviews the query and provides the correct answer
- The response is delivered back to the user

Benefits of HITL

- Improves overall reliability of the system
- Handles edge cases and complex scenarios effectively
- Builds user trust through guaranteed response quality

Limitations

- Introduces additional delay in response time
- Requires dedicated human resources
- May not scale easily under high query volume